

Chapter 9

Planning a Release

Most software projects will do best with a new release every two to six months. Certain website projects may release even more frequently, but even then there is a benefit to collecting related new features into a release. It is frequently useful to start release planning from a product development roadmap showing the main areas of focus for the next handful of new releases. This product development roadmap will certainly change—and we want it to since the changes will be the result of our learning more about our product, its market, and our ability to develop the product.

A product development roadmap can be as simple as a list of the main areas of focus, or *themes* as Kent Beck calls them, for each of the next few releases. For example, for the next release of the BigBucksJobs.com website we may list the following themes:

- resume filtering and screening tools for companies
- automated search agents for job seekers
- improved query performance

Starting with a rough product development roadmap, there are two questions we use to initiate release planning:

- When do we want the release?
- What is the priority of each story?

Once we have the answers to these questions we plan the release by estimating how much work the team will be able to accomplish in each iteration. Using this estimate of how much work we can do in an iteration, we make a reasonable prediction about how many iterations it will take to produce a release that meets the customer's expectations.

When Do We Want the Release?

Ideally, the developers and the customer can talk about a range of dates, rather than a specific date: “We’d like to release in May, but as long as we release sometime in July, that’s fine.” An iterative, story-driven process makes it easy to fix a date but difficult to fix what will be included by a given date. If a team can start release planning with a range of acceptable dates they will have more flexibility in timing releases. For example, starting with a date range in mind enables a team to make statements like “After six or seven iterations we should have the minimum functionality and maybe ten to twelve before we have everything on the 1.0 wish list.”

In some cases the date truly is fixed. Most commonly this occurs when preparing a release for a trade show, a key customer release, or some similar milestone. If this is the case, release planning is actually a bit easier as there are fewer variables to consider. However, the decisions about which stories to include will usually be more difficult.

What Would You Like in It?

In order to plan a release, the customer must prioritize the stories. Prioritizing stories into the familiar high, medium and low categories is useful but can lead to tedious arguments over what constitutes a high priority story as opposed to a medium priority story. Fortunately, we can borrow a technique from DSDM, another of the agile processes.¹

DSDM includes a prioritization technique referred to as the MoSCoW rules. MoSCoW is an acronym for

- Must have
- Should have
- Could have
- Won’t have this time

The must-have features are those that are fundamental to the system. Should-have features are important but there’s a short-term workaround for

1. For information on DSDM see *DSDM: Business Focused Development* (Stapleton 2003).

them. If the project has no time constraints, the should-have features would normally be considered mandatory. Could-have features are ones that can be left out of the release if time runs out. Features prioritized as won't-have are ones that are desired but acknowledged as needing to come in a later release.

Prioritizing the Stories

There are many dimensions along which we can sort stories. Among the technical factors we can use are:

- the risk that the story cannot be completed as desired (for example, with desired performance characteristics or with a novel algorithm)
- the impact the story will have on other stories if deferred (we don't want to wait until the last iteration to learn that the application is to be three-tiered and multi-threaded)

Additionally, customers and users have their own set of factors they could use to sort the stories, including the following:

- the desirability of the story to a broad base of users or customers
- the desirability of the story to a small number of important users or customers
- the cohesiveness of the story in relation to other stories (for example, a “zoom out” story may not be high priority on its own but may be treated as such because it is complementary to “zoom in,” which is high priority)

Collectively, the developers have a sequence in which they would like to implement the stories, as will the customer. When there is a disagreement to the sequence, the customer wins. Every time.

However, customers cannot prioritize without some information from the development team. Minimally, a customer needs to know approximately how long each story will take. Before the stories are prioritized, they have already been estimated and the estimates written on the story cards, as shown in Story Card 9.1.

At this point the customer does not sum the estimates and make decisions about what will or won't fit in a release. Instead, she uses the estimates, along with her own assessment of the value of each story, to sort the stories so that they maximize the value delivered to the organization. A particular story may

be highly valuable to the organization but will take a month to develop. A different story may only be half as valuable but can be developed in a day.

The site always tells a shopper what the last 3 (?) items she viewed are and provides links back to them. (This works even between sessions.)

Estimate: 3 days

- Story Card 9.1 Provide links back to previously viewed items.

Cost Changes Priority

A few years ago my team was building a Windows user interface for a customer who was transitioning a large application from an old DOS-based system. In the DOS system the Enter key was used to move forward between fields. The customer wanted us to do the same in her new Windows system. From her customer perspective it was logical that it would take the same amount of development time to use either Enter or Tab. However, we estimated that it would take about an extra person week to use the Enter key. After hearing that, our customer quickly lowered her priority on that story. It was a high priority to her when she thought it was a few hours; when it was a week, there were many other things she decided she would rather have.

Mixed Priorities

If a customer is having trouble prioritizing a story, the story may need to be split. Splitting a story allows the customer to prioritize the separate stories differently. On one project I had the story description shown in Story Card 9.2. The customer struggled to prioritize the story because searching by author and title were essential, while the other search fields were considered nice but were not essential. The story was split into three: one story for searching by author or title, another for searching by publication name or date, and a third allowing for the criteria to be combined.

Users can search for magazine articles by author, publication name, title, date or any combination of these.

■ Story Card 9.2 Search criteria.

Risky Stories

Looking back over earlier approaches to software development, it is clear that there has been an ongoing debate about whether a project should first go after the riskiest parts or the most valuable parts of the project. Probably the leading proponent of risk-driven development has been Barry Boehm whose spiral model focuses on the early elimination of risk (1988). On the other end has been Tom Gilb who advocates doing the “juicy bits” first (1988).

Agile approaches are firmly in the camp of doing the juicy bits first. This allows agile projects to avoid solving risks too far in advance and allows them to defer building infrastructural code that may not be needed. Favoring the juicy bits also makes it possible for a project to release early, when only the highest-valued functionality is available.

But, even when going after the juicy bits first, we still need to consider risk when prioritizing stories. Many developers have a tendency to want to do the riskiest stories first. Sometimes this is appropriate but the decision must still be made by the customer. However, the customer considers input from the technical team when prioritizing the stories.

On one recent project in the biotech space some of the stories called for novel extensions to a standard statistical approach called expectation maximization. Because the work being undertaken was truly new, the team could not be sure if it could be accomplished at all or how long it would take. The product would still have been saleable without the inclusion of these stories, so the customer prioritized them to be near the middle of the stack. However, once the customer was made aware of the extremely high risk associated with these stories, enough of them were given higher priorities in order to determine what was involved in developing the novel algorithms.

Prioritizing Infrastructural Needs

Frequently the risky stories are associated with infrastructural or nonfunctional needs such as performance. I was on a project to develop a web program that

would display charts of stock prices. One of our stories is shown in Story Card 9.3. For the baseline web server machinery that had been specified, this level of performance could be a significant challenge. The difficulty of meeting this performance requirement would have a profound impact on our architectural decisions.

Be able to generate 50 stock chart images per second.

■ Story Card 9.3 Generate 50 images per second.

We'd already committed to Java as our server-side language but could we achieve 50 images per second with Java? Would we instead need to go with native C or C++ code for the image generation? Or could we achieve our throughput goal with a strong caching algorithm that would serve up the same chart for requests that were only seconds apart?

In this case the customer had written Story Card 9.3 for us. However, she prioritized it fairly low. Our first few iterations would be targeted at developing features that could be shown to prospects and used to generate initial sales and interest in the product. Our customer reasoned that we could always add scalability in later. In some cases it is easy to refactor a system to improve its scalability. In other cases, that type of refactoring can be very difficult. It is up to the developers to help the customer by identifying stories that can be deferred but may become much more costly to develop if implemented later. Developers must not, however, take this as a license to steer the customer toward early implementation of their favorite technical features.

On another project, the customer clearly wanted the application to be deployable as a three-tiered application with a database server, a client machine, and a middle-tier that would route requests and data between them. The customer had talked with the team about this in various meetings and the marketing literature she was preparing described the system as three-tiered. However, none of the stories was written so that it required the addition of the middle tier.

This was becoming troubling to the technical team. They did not mind starting with a simple two-tiered system (database server and client machine) but after a couple of iterations had gone by they became more and more concerned that the middle tier had not been added. They knew that adding the middle tier would still be easy but that it would get a little harder with each iteration. Also, because the user stories were written with their focus entirely on end-user functionality it was not clear when such an infrastructural need would be added.

The solution was to write a story that made the three-tier capability a higher priority to the customer who was prioritizing the team's work. In this case we added the story: “During installation the user can decide to install everything locally on her PC or to install the client, middle-tier and server separately.”

Selecting an Iteration Length

Collectively the developers and the customer select an iteration length that will work for them. Iteration lengths are typically from one to four weeks. Short iterations allow for more frequent course corrections of the project and more visibility into its progress; however, there is a slight overhead associated with each iteration. Err on the side of iterations that seem a little too short rather than ones that seem a little too long.

As much as possible, stick with a constant iteration length for the duration of the project. With consistent iterations, projects fall into a natural rhythm that can be beneficial to the pace of the team. Naturally there will be times when you need to alter the iteration length. For example, a team that has been using three-week iterations is asked to prepare the next version for an important tradeshow in eight weeks. Rather than stopping after two three-week iterations with two weeks left before the show, they can start with two normal three-week iterations and then follow those with an abbreviated two-week iteration. There's nothing wrong with this. What you want to avoid is random changes to the iteration length.

From Story Points to Expected Duration

Suppose that the customer has prioritized all of the story cards. The team sums the estimates from each card and comes up with 100 story points. Using story points made it easier for estimating the stories, but now we need a way to convert story points into a predicted duration for the project.

The answer, of course, is to use velocity. As we learned in Chapter 8, “Estimating User Stories,” velocity represents the amount of work that gets done in an iteration. Once we know a team's velocity, we can use it to turn ideal days into calendar days. For example, if we estimate our project at 100 ideal days, then with a velocity of 25 we can estimate that it will take $100 / 25 = 4$ iterations to complete the project.

The Initial Velocity

There are three ways to get an initial value for velocity:

1. Use historical values.
2. Run an initial iteration and use the velocity of that iteration.
3. Take a guess.

Using historical values is the best option, but it is only viable if we have an existing team that is rolling off a project similar to the new project and if no one is joining or leaving the team. Unfortunately, it is rare that the exact same team gets to work on two consecutive similar projects.

Running an initial iteration is a great way to get a starting velocity. However, there are many times when this is not viable. For example, suppose your boss comes to you with a new product idea. She's written the user stories she thinks are needed in the first version. She's used those stories to do market research and thinks the product will earn \$500,000 the first year. If the product can be developed cheaply enough, the company will pursue it. If not, they'll pass. When your boss asks you what it will cost to develop, you are not always given the freedom to say, "Let me run a sample iteration for two weeks and get back to you." In cases like this you need a way to take a guess at the velocity.

Guessing at Velocity

If we need to guess at velocity we should at least do so in a way that makes sense when we explain it to others. Fortunately, there's a reasonable way of doing this if you followed the advice of Chapter 8, "Estimating User Stories," and defined a story point as approximately one ideal day of work.

If a story point is one ideal day of work, we can estimate initial velocity by estimating how many actual days it will take us to complete a full ideal day of work. Over the course of an iteration it is clear the team will have many interruptions that will prevent the team from having ideal days. Their actual days will differ from ideal days because of time spent answering email, phone calls, all-company meetings, departmental meetings, training, giving or attending demos, washing the boss' car, interviewing new candidates, illnesses, vacation, and so on. Because of all of these interruptions it is common to start with an expected velocity that is somewhere between one-third and one-half the number of developer-days in an iteration. For example, a six-person team using two-week (ten working day) iterations would have sixty developer-days each

iteration. They may want to estimate velocity between 20 and 30 story points per iteration depending on how different they expect their days to be from ideal days.

Of course, as the project progresses through the first few iterations the team will gain a much better feeling for the duration of the project. They'll know within an iteration or two how far off their velocity estimate was and will be able to refine the estimate and communicate the plan with more confidence.

Creating the Release Plan

So, if the project has 100 story points and we estimate velocity at 20 story points per iteration, then we can expect it to take five iterations. The final step in release planning is to allocate stories into each of the iterations. Collaboratively the customer and the developers select twenty story points worth of the highest priority stories and place them into the first iteration. The next twenty story points go into the second iteration and so on until all the stories have been allocated.

Depending on whether the team is entirely collocated (including stakeholders such as upper managers) and the organization's need for formality, there are many ways to communicate a release plan. For example, I've used each of the following:

- With a collocated team, I've pinned story cards to the wall using columns to indicate the iterations.
- With stories in a spreadsheet, I've sorted the stories based on their iterations and then drawn a thick bold line after the last story in each iteration.
- For interested remote stakeholders, I've photocopied notecards (three to a page, or six to a page if you reduce the size). I indicate the start of each iteration and add a nice cover page.
- For interested, high ceremony, remote stakeholders, I've created trivial Gantt charts. I create entries like "Iteration 1" and then list as subordinate to it the story names for that iteration.

A Warning

Be careful not to put too much faith in the release plan. The techniques described in this chapter will help you estimate the approximate duration of a project and allow you to make statements like “The product will be ready for release in approximately 5–7 iterations.” They don’t give you enough precision, however, to say things like “We’ll be done on June 3.”

Use the release plan to set initial expectations but then constantly reset those expectations as you gain new information. Monitor each iteration’s velocity and re-estimate the stories whenever you learn something new that affects the estimates.

Summary

- Before planning a release it is necessary to know approximately when the customer would like the release and the relative priorities of the stories.
- Stories should be prioritized into a specific order (first, second, third and so on) rather than into groups (very high, high, medium and so on).
- Stories are prioritized by the customer but with input from the developers.
- Estimates, which are in ideal days, are converted into calendar time using velocity.
- It is often necessary to estimate a team’s initial velocity.

Developer Responsibilities

- You are responsible for providing information (sometimes including your underlying assumptions and possible alternatives) to the customer in order to help her prioritize the stories.
- You are responsible for resisting the urge to prioritize infrastructural or architectural needs higher than they should be.
- You are responsible for creating a release plan that is built on realistic estimates yet includes an appropriately sized project buffer.

Customer Responsibilities

- You are responsible for prioritizing the user stories into the precise order you value them. It is not sufficient to sort them into stacks of high, medium and low priority.
- You are responsible for expressing your honest deadlines for the release. If you need it on July 15th, don't tell the developers you need it on June 15th just to be safe.
- You are responsible for understanding the difference between ideal time and calendar time.
- You are responsible for splitting stories that contain components that you want prioritized differently.
- You are responsible for understanding why a programmer with a personal velocity of 0.6 should not be reprimanded or criticized because her velocity is less than 1.0.

Questions

- 9.1 What are three ways of estimating a team's initial velocity?
- 9.2 Assuming one-week iterations and a team of four developers, how many iterations will it take the team to complete a project with 27 story points if they have a velocity of 4?

This page intentionally left blank